# Smart Budgeting and Financial Goal Management System

*Software Requirements Specification (SRS)*

| | |
|---|---|
| **Project** | Smart Budgeting and Financial Goal Management System — Smart Personal Budgeting and Financial Goal Management System |
| **Prepared for** | Team J |
| **Version / Date** | Version 1.0 \| 8 October 2026 |

*Working draft for project implementation. Items marked "decision required" should be confirmed by the team before development.*

# Document control

| Field | Details |
|---|---|
| Purpose | Define agreed scope, functional requirements, quality requirements and acceptance criteria. |
| Audience | Team J, supervisor/lecturer, developers, testers and project stakeholders. |
| Status | Baseline draft; confirm open decisions before implementation. |
| Source basis | Team J project idea presentation: problem, proposed solution, six feature areas and expected benefits. |

# 1. Introduction

## 1.1 Purpose

This SRS describes the requirements for Smart Budgeting and Financial Goal Management System, a digital personal-finance platform that helps an individual record income and expenses, create category budgets, set savings goals, review financial summaries, and receive spending insights.

## 1.2 Problem statement

People may struggle to track income and expenses, save consistently, stay within budgets, achieve financial goals, and understand spending patterns. This can contribute to overspending and missed financial targets.

## 1.3 Objectives

- Make recording income and expenses straightforward.
- Help users plan category-level spending limits.
- Track progress towards savings goals.
- Show clear summaries and visual reports.
- Offer understandable, data-based tips that support better decisions.

## 1.4 Scope

In scope for the proposed first release:

- User registration, sign-in and personal account management.
- Income and expense records with date, amount, category and notes.
- Budget creation and comparison of actual spending against limits.
- Savings goals with target amount, target date and progress tracking.
- Dashboard, charts and date-filtered reports.
- Rule-based smart insights based on recorded data.
- Basic settings, validation, privacy and secure access.

Out of scope unless the team explicitly approves later: direct bank/mobile-money integrations, automatic transaction imports, payment transfers, lending/investment advice, multi-currency conversion, shared household accounts, and AI-generated financial advice.

### 1.5 Intended users

| User / stakeholder | Needs |
|---|---|
| Registered user | Manage personal financial records, budgets, goals and reports. |
| Project administrator (optional for MVP) | Manage technical configuration and handle support issues without casually viewing user financial details. |
| Development team | Build, test, deploy and maintain the system. |
| Supervisor / evaluator | Assess whether requirements are implemented and demonstrated. |

# 2. Product overview

Smart Budgeting and Financial Goal Management System is intended to act as a personal financial assistant. The first release assumes a user manually enters financial records. The system calculates totals and progress from stored records; it does not independently verify that entered figures are true.

# 3. Functional requirements

| ID | Requirement | Description |
|---|---|---|
| FR-01 | Account registration | The system shall allow a new user to create an account using the agreed unique identifier and password. |
| FR-02 | Authentication | The system shall allow a registered user to sign in and sign out securely. |
| FR-03 | Profile/settings | The system shall allow a user to view and update permitted profile and preference fields. |
| FR-04 | Record income | The system shall allow a user to create, view, edit and delete an income record containing amount, date, source/category and optional note. |
| FR-05 | Record expenses | The system shall allow a user to create, view, edit and delete an expense record containing amount, date, category and optional note. |
| FR-06 | Categories | The system shall provide default categories and allow users to manage personal categories if approved for MVP. |
| FR-07 | Budgets | The system shall allow a user to set a spending limit for a category and a defined period. |
| FR-08 | Budget monitoring | The system shall calculate spending against each budget and |

| | | |
|---|---|---|
| | | show remaining amount or overspend. |
| FR-09 | Savings goals | The system shall allow a user to create, edit, pause/complete and delete a goal with a name, target amount and optional target date. |
| FR-10 | Goal progress | The system shall display progress towards a savings goal based on contributions recorded by the user. |
| FR-11 | Dashboard | The system shall show a summary of income, expenses, balance/net cash flow, budgets and savings goals for a selected period. |
| FR-12 | Reports | The system shall display date-filtered summaries and charts by category and over time. |
| FR-13 | Smart insights | The system shall generate understandable rule-based tips from available records, such as unusually high category spending or repeated budget overruns. |
| FR-14 | Data isolation | The system shall ensure a user can only access their own financial records and goals. |
| FR-15 | Validation | The system shall reject invalid amounts, dates, missing required fields and malformed inputs with clear messages. |
| FR-16 | Data persistence | The system shall retain saved records after a user signs out or refreshes the application. |

# 4. Non-functional requirements

| ID | Requirement |
|---|---|
| NFR-01 Security | Passwords must be stored using a suitable one-way password hash; never as plain text. Authenticated routes and server-side ownership checks are required. |
| NFR-02 Privacy | Financial information must not be exposed to other users. Collect only information needed for the agreed features. |
| NFR-03 Usability | Core actions should be understandable to a first-time user; forms should provide clear labels, validation and feedback. |
| NFR-04 Performance | For a typical student-project dataset and normal network conditions, common pages and summaries should respond within 3 seconds in at least 95% of test runs (target to validate). |

| | |
|---|---|
| NFR-05 Reliability | A successful save should persist to the database and be visible after reload. Errors must not silently discard data. |
| NFR-06 Compatibility | The responsive interface should support current mainstream desktop and mobile browsers. |
| NFR-07 Maintainability | Use modular code, meaningful names, migrations, documented setup steps and version control. |
| NFR-08 Accessibility | Use readable contrast, keyboard-operable controls, labels and text alternatives for meaningful charts. |
| NFR-09 Backup | The team should define a database backup and restore procedure before production deployment. |
| NFR-10 Financial disclaimer | Insights must be educational summaries, not guarantees or regulated financial advice. |

# 5. Business rules and calculations

- Amounts must be greater than zero for income, expenses, budget limits and goal targets; corrections are made by editing or deleting records.
- Income total = sum of income records within the selected period.
- Expense total = sum of expense records within the selected period.
- Net cash flow = income total − expense total. This is a period summary, not necessarily a bank balance.
- Budget used = sum of expenses matching the budget category and period. Remaining = budget limit − budget used; negative remaining indicates overspending.
- Goal progress percentage = recorded contributions ÷ target amount × 100, capped at 100% for the progress bar; preserve actual contribution totals separately.
- Budget period options (weekly/monthly/custom) and the meaning of goal contributions are decision-required and must be fixed before coding.

# 6. Data requirements

The proposed core entities are User, Category, Transaction, Budget, SavingsGoal and GoalContribution. Transaction type distinguishes income from expense. Every user-owned record must have an ownership relationship and be checked on every read or write.

# 7. User interface requirements

- Provide sign-in and registration screens.
- Provide a dashboard with summary cards, charts and shortcuts to add a transaction.
- Provide transaction list, add/edit forms and date/category filters.
- Provide budget list/progress indicators and goal progress views.
- Provide reports with a selected date range and readable labels.
- Use consistent navigation, responsive layout and clear empty/error states.

# 8. Acceptance criteria

- A new user can register, sign in and sign out.
- A user can add, edit and delete income and expense records; totals update correctly.
- A budget displays spending and remaining amount calculated from matching expenses.
- A savings goal shows target, recorded contributions and progress correctly.
- Dashboard and reports agree with the underlying records for the same period.
- A user cannot view or change another user's records by changing a URL or request identifier.
- Invalid form submissions are rejected with helpful messages.
- The app can be installed and run using the project setup guide.

# 9. Assumptions, constraints and decisions

| Topic | Current assumption / decision required |
|---|---|
| Platform | Responsive web application is proposed; confirm if mobile app is expected. |
| Technology | Suggested student-project stack: React + TypeScript frontend, Django REST Framework backend, PostgreSQL database; team approval required. |
| Currency | ZMW (Zambian Kwacha) is a reasonable default for the first release; confirm whether multiple currencies are required. |
| Authentication | Email or username login is undecided; choose one before schema and UI implementation. |
| Insights | Rule-based insights for MVP; no AI dependency required. |
| Savings contribution | Decide whether contributions are entered separately or represented as transfers/transactions. |
| Budget periods | Choose calendar month only for simpler MVP, or weekly/monthly/custom. |
| Deployment | Hosting provider, domain, backup and operating costs are not yet specified. |

# 10. Risks

- Scope creep from bank integrations or advanced AI features.
- Incorrect calculations if period boundaries and categories are unclear.
- Exposure of sensitive financial data if ownership checks are missed.
- Users may interpret a calculated cash-flow figure as a verified bank balance.
- Insufficient time for testing, deployment and documentation.